STACKLY RESERVE

LUXURY RESTAURANT RESERVATION PLATFORM

Technical Documentation

Project Name: Stackly Reserve – Luxury Restaurant Reservations & Haute Gastronomy Platform

Project Type: Restaurant Reservation and Fine Dining Management Platform

Architecture: Static Multi-Page Frontend with Client/Admin Portals

Frontend: HTML5, CSS3, Vanilla JavaScript ES6

CSS Framework: Tailwind CSS CDN + Custom CSS

Icon Library: Lucide Icons

External Libraries: Canvas Confetti

Data Storage: JavaScript Data Objects + Browser LocalStorage

Image Format: WebP

Deployment: GitHub Pages / Static Web Hosting

Version: 1.0

Document Version: 1.0

Prepared By: Dhamodharan P

---

1. PROJECT OVERVIEW

1.1 Project Description

Stackly Reserve is a luxury restaurant reservation and haute gastronomy platform designed to provide users with a premium dining discovery and reservation experience.

The platform allows users to:

- Discover fine dining restaurants.
- Search and filter restaurants.
- Explore Michelin-starred restaurants.
- View restaurant profiles.
- Explore menus and dining experiences.
- Select seating areas and tables.
- Make restaurant reservations.
- Save restaurants to a personal hotlist.
- Manage reservations.

- View digital VIP reservation passes.
- Contact a concierge service.
- Manage user profiles.
- Access an administrative restaurant operations console.

The application uses a dark luxury-themed user interface with gold/amber accents and responsive layouts. The frontend is implemented using HTML5, Tailwind CSS, custom CSS, and Vanilla JavaScript.

The application is designed as a frontend prototype with browser-based persistence using LocalStorage rather than a production backend/database.

---

2. PROJECT OBJECTIVES

The main objectives of the project are:

- Build a premium restaurant discovery platform.
- Provide an interactive restaurant reservation experience.
- Display Michelin-starred restaurants and luxury dining destinations.
- Allow users to search and filter restaurant listings.
- Provide detailed restaurant information.
- Provide interactive table and seating selection.
- Allow users to save favorite restaurants.
- Manage upcoming and historical reservations.
- Provide a digital VIP reservation pass.
- Provide a concierge communication interface.
- Provide client profile management.
- Provide a Maître D’ administrative dashboard.
- Implement responsive design for desktop, tablet, and mobile devices.
- Maintain reusable frontend components.
- Provide persistent prototype data using LocalStorage.

---

3. TECHNOLOGY STACK

3.1 HTML5

HTML5 is used for the structural and semantic foundation of the platform.

HTML is used for:

- Page layouts
- Navigation
- Restaurant cards
- Forms
- Reservation interfaces
- Authentication
- Profile pages
- Dashboard layouts
- Tables
- Modals
- Footer sections

3.2 Tailwind CSS

The project uses Tailwind CSS through CDN.

```
<script src="https://cdn.tailwindcss.com"></script>
```

Tailwind is used extensively for:

- Responsive layouts
- Flexbox
- Grid
- Typography
- Spacing
- Borders
- Shadows
- Colors
- Buttons
- Cards
- Modal interfaces
- Dashboard components

3.3 Custom CSS

Custom styles are maintained in:

```
css/styles.css
```

The stylesheet provides:

- Global design tokens
- Custom animations
- Background effects
- Scrollbar styling
- Component-specific styling
- Responsive behavior
- Luxury visual effects

3.4 JavaScript ES6

The platform uses Vanilla JavaScript without a frontend framework.

JavaScript provides:

- Dynamic content rendering
- Authentication
- Reservation management
- Restaurant filtering
- Search
- Favorites
- Modals
- Dashboard rendering
- LocalStorage persistence
- Toast notifications
- Concierge interactions
- Interactive floor plans
- Table management

---

4. EXTERNAL LIBRARIES

4.1 Lucide Icons

The platform uses Lucide for interface icons.

```
<script src="https://unpkg.com/lucide@latest"></script>
```

Icons are used for:

- Navigation

- Search
- Calendar
- Clock
- User profile
- Restaurant information
- Table management
- Dashboard controls
- Reservation status

4.2 Canvas Confetti

Canvas Confetti is used to provide visual celebration effects after selected successful user actions.

```
<script src="https://cdn.jsdelivr.net/npm/canvas-
confetti@1.9.4/dist/confetti.browser.min.js"></script>
```

4.3 Google Fonts

The application uses:

- Cinzel
- Cormorant Garamond
- Plus Jakarta Sans

These fonts establish the luxury editorial appearance of the platform.

---

5. PROJECT STRUCTURE

```
Restaurant Reservation Platform/

├── index.html
├── restaurants.html
├── restaurant-detail.html
├── about.html
├── concierge.html
├── my-reservations.html
├── login.html
├── signup.html
├── client-profile.html
├── admin-dashboard.html
```

```
├── 404.html
│
├── css/
│   └── styles.css
│
├── js/
│   ├── components.js
│   ├── data.js
│   ├── main.js
│   ├── store.js
│   │
│   └── pages/
│       ├── home.js
│       ├── restaurants.js
│       ├── restaurant-detail.js
│       ├── my-reservations.js
│       ├── client-profile.js
│       ├── concierge.js
│       └── admin-dashboard.js
│
├── assets/
│   ├── asset-01.webp
│   ├── asset-02.webp
│   ├── ...
│   └── asset-53.webp
│
└── .github/
    └── workflows/
        └── static.yml
```

---

6. PAGE ARCHITECTURE

The project contains 11 primary HTML pages.

Page	Purpose
index.html	Home / Landing Page
restaurants.html	Restaurant Catalog

restaurant-detail.html	Restaurant Profile & Seating
about.html	Platform Philosophy
concierge.html	VIP Concierge
my-reservations.html	Reservation Management
login.html	User/Admin Authentication
signup.html	VIP Membership Registration
client-profile.html	Client Profile
admin-dashboard.html	Restaurant Operations Console
404.html	Error Page

---

7. HOME PAGE

File

```
index.html
```

The home page is the main entry point for Stackly Reserve.

Main Features

- Luxury hero section
- Restaurant discovery
- Featured dining destinations
- Michelin restaurant highlights
- Dining experiences
- Reservation call-to-actions
- Concierge promotion
- Journal/articles
- Premium visual presentation
- Global navigation
- Footer

The home page uses `home.js` for page-specific rendering and interaction.

---

8. RESTAURANT CATALOG

File

```
restaurants.html
```

The restaurant catalog allows users to discover and filter restaurants.

Features

- Restaurant search
- City filtering
- Cuisine filtering
- Michelin-star filtering
- Price filtering
- Sorting
- Grid view
- List view
- Map-oriented view
- Restaurant cards
- Urgent table availability
- Restaurant detail navigation

---

9. RESTAURANT FILTERING

The catalog filtering system supports multiple criteria.

Search

Search can match:

- Restaurant name
- Cuisine
- City
- Chef
- Ambience tags

Michelin Filter

Supported values include:

- All
- Michelin starred
- 2 Michelin stars
- 3 Michelin stars

Cuisine Filter

Users can filter restaurants according to cuisine type.

Price Filter

Restaurants can be filtered using their configured price range.

Sorting

Supported sorting options include:

- Recommended
- Michelin Stars
- Rating
- Price High to Low
- Price Low to High

---

10. RESTAURANT DETAIL PAGE

File

```
restaurant-detail.html
```

The restaurant detail page provides an immersive restaurant profile.

The selected restaurant is determined from the URL query parameter:

```
restaurant-detail.html?id=<restaurant-id>
```

Features

- Restaurant gallery
- Hero image
- Michelin rating

- Awards
- Restaurant address
- Cuisine information
- Price range
- Customer rating
- Review count
- Ambience tags
- Dietary options
- Seating areas
- Interactive floor plan
- Table selection
- Menu information
- Reservation workflow
- Favorite/hotlist functionality

---

11. INTERACTIVE FLOOR PLAN

The restaurant detail page includes an interactive seating/floor-plan system.

Users can select tables based on:

- Table number
- Seating area
- Capacity
- Availability
- Table type
- Special table tag

Examples of seating areas include:

- Chef Counter
- Skyline Window
- Main Dining
- Wine Vault

The selected table is tracked through:

```
selectedTableId
```

---

12. RESERVATION SYSTEM

The reservation system is one of the core features of the application.

Users can select:

- Restaurant
- Date
- Time
- Party size
- Seating area
- Table
- Guest information
- Special requests

Reservations are maintained through the central application store.

Reservation Data

Reservation records contain information such as:

```
Reservation ID
Restaurant ID
Restaurant Name
Restaurant Image
Date
Time
Guest Count
Table Number
Seating Area
Confirmation Code
Guest Information
Reservation Status
```

---

13. MY RESERVATIONS

File

```
my-reservations.html
```

The reservation management page provides three major views:

```
Upcoming Tables
Saved Hotlist
Dining History
```

Upcoming Reservations

Displays confirmed reservations.

Users can view:

- Restaurant
- Location
- Date
- Time
- Guest count
- Table
- Confirmation code
- Digital VIP pass

Users can also cancel a reservation.

Saved Hotlist

Displays restaurants saved by the user.

Favorites are stored using:

```
stackly_favorites
```

Dining History

Displays completed and cancelled reservations.

---

14. DIGITAL VIP PASS

The application provides a digital VIP reservation pass.

The pass can contain:

- Restaurant name
- Guest name
- Reservation date
- Reservation time

- Table number
- Confirmation code
- Seating area
- QR-code style interface

The pass is accessed through the reservation management interface.

---

15. FAVORITES / HOTLIST

Users can save restaurants to their personal hotlist.

The favorite state is managed through:

```
store.toggleFavorite()
```

Favorite restaurant IDs are persisted in:

```
stackly_favorites
```

The restaurant detail page also provides a Save / Saved to Hotlist interaction.

---

16. AUTHENTICATION MODULE

The platform provides:

```
login.html
signup.html
```

16.1 Login

The login system supports:

- Email
- Password
- Client login
- Admin login
- Role verification
- Authentication feedback
- Persistent session state

16.2 Registration

Users can create a new account by providing:

- Name
- Email
- Password
- Phone
- Additional profile information

The system checks for duplicate email addresses.

---

17. DEMO USER ROLES

The application defines two primary account types:

Client

Role:

```
client
```

Profile type:

```
Black Diamond VIP Gastronome
```

Admin

Role:

```
admin
```

Profile type:

```
Director of Restaurant Operations & Maître D’
```

---

18. CENTRAL STATE STORE

The main application state is implemented through:

```
js/store.js
```

The primary class is:

```
class StacklyStore
```

The store acts as a central state-management layer for the frontend.

Store Responsibilities

The store manages:

- Accounts
- Current authenticated user
- Reservations
- Favorites
- Concierge inquiries
- Table statuses
- Filters
- Modal state
- Toast notifications
- Booking state

---

19. REACTIVE STORE

The store implements a simple subscription mechanism.

```
subscribe(listener)
```

and:

```
notify()
```

Pages subscribe to state updates so that the UI can be refreshed when application state changes.

This provides a lightweight reactive architecture without using React, Vue, Angular, or another frontend framework.

---

20. LOCAL STORAGE

The platform uses browser LocalStorage for client-side persistence.

Important keys include:

```
stackly_accounts
stackly_auth_user
stackly_logged_out
stackly_concierge_inquiries
```

```
stackly_reservations
stackly_favorites
```

Stored Information

LocalStorage is used for:

- Account records
- Login state
- Current user
- Reservations
- Favorite restaurants
- Concierge inquiries

---

21. RESTAURANT DATA MODEL

Restaurant information is maintained in:

```
js/data.js
```

The main dataset is:

```
RESTAURANTS_DATA
```

Restaurant records include:

```
id
name
tagline
slug
cuisine
michelinStars
awards
priceRange
pricePerPerson
neighborhood
city
country
address
latitude
longitude
rating
```

```
reviewCount
heroImage
gallery
ambienceTags
dietaryOptions
featured
trendingToday
urgentTablesCount
seatingAreas
tables
```

---

22. DINING EXPERIENCES

The application contains a dedicated experience dataset:

```
EXPERIENCES_DATA
```

The current project contains four curated luxury dining experiences.

Each experience contains information such as:

- Experience title
- Restaurant
- Location
- Date/time
- Price per person
- Duration
- Image
- Tags
- Description
- Highlights
- Curator
- Remaining spots

---

23. JOURNAL / ARTICLE MODULE

The platform includes:

```
JOURNAL_ARTICLES
```

The dataset contains three editorial articles.

Article information includes:

- Title
- Subtitle
- Category
- Reading time
- Publication date
- Author
- Image
- Excerpt

---

24. CONCIERGE MODULE

File

```
concierge.html
```

The concierge page provides a premium support experience.

Features include:

- VIP concierge request form
- Chat interface
- Dining requirement submission
- Inquiry tracking
- Concierge response simulation

Concierge Store

Concierge requests are stored using:

```
stackly_concierge_inquiries
```

Each inquiry receives a generated reference ID.

---

25. CLIENT PROFILE

File

```
client-profile.html
```

The client profile represents the user's private VIP dining identity.

Profile information includes:

- Name
- Email
- Phone
- City
- Membership tier
- Loyalty points
- Dining preferences
- Dietary notes
- Favorite cuisines
- Diner since date
- Profile avatar

---

26. ADMIN DASHBOARD

File

```
admin-dashboard.html
```

The Admin dashboard is presented as the:

```
Maître D’ Command Room
```

It is designed for restaurant operations management.

---

27. ADMIN DASHBOARD FEATURES

The administrative console contains sections for:

- Overview
- Floor Management
- Reservations
- Cellar
- Kitchen

The dashboard also includes:

- Live service clock
- Restaurant selector
- Reservation filters

- Search
- Guest dossier
- Walk-in functionality
- Table status management
- Operational information

---

28. ADMIN RESTAURANT SELECTION

The admin interface supports restaurant selection.

Configured restaurants include:

- L’Aura Élégante – New York
- Sushi Shibata Ginza – Tokyo
- Il Giardino Sul Mare – Amalfi Coast
- Kallio & Måne – Stockholm
- Asador del Oro – Dubai
- Château de Vignoble – Bordeaux

---

29. ADMIN RESERVATION MANAGEMENT

Administrators can filter reservations by status.

Supported statuses include:

```
all
confirmed
seated
completed
cancelled
```

Administrators can also search reservations.

---

30. VIP GUEST DOSSIER

The admin dashboard provides a guest dossier modal.

The dossier displays:

- Guest name
- Email
- Phone

- Confirmation code
- Reservation date
- Reservation time
- Table
- Seating area
- Special dining directives

This provides restaurant staff with a consolidated view of VIP guest information.

---

31. TABLE STATUS MANAGEMENT

The store maintains table states such as:

```
available
reserved
seated
vip_hold
cleaning
```

The initial table state configuration is stored in:

```
store.tableStatuses
```

This supports real-time-style restaurant floor management within the frontend prototype.

---

32. COMPONENT ARCHITECTURE

Global reusable components are implemented in:

```
js/components.js
```

The component architecture is used for common UI elements including:

- Navigation
- Footer
- Booking modal
- Reservation pass modal
- Review modal
- Search modal
- Other shared interfaces

This reduces duplicated HTML across multiple pages.

---

33. MAIN APPLICATION CONTROLLER

The global application behavior is managed by:

```
js/main.js
```

The main script is responsible for common functionality such as:

- Page initialization
- Preloader behavior
- Scroll animations
- Shared event handling
- Global UI interactions

---

34. PAGE-SPECIFIC CONTROLLERS

The project separates page-specific functionality into controllers.

```
js/pages/
```

home.js

Controls Home page behavior.

restaurants.js

Controls:

- Restaurant catalog
- Search
- Filters
- Sorting
- Catalog views

restaurant-detail.js

Controls:

- Restaurant details
- Gallery
- Table selection
- Seating
- Booking interaction
- Favorite state

my-reservations.js

Controls:

- Upcoming reservations
- Saved restaurants
- Dining history

client-profile.js

Controls client profile functionality.

concierge.js

Controls concierge interactions.

admin-dashboard.js

Controls administrative operations.

---

35. MODAL ARCHITECTURE

The platform makes extensive use of modal-based interactions.

Examples include:

- Booking modal
- Search modal
- Review modal
- Reservation pass
- Guest dossier
- Walk-in interface
- Restaurant details

Modal state is managed through the central store.

---

36. TOAST NOTIFICATION SYSTEM

The platform provides user feedback using toast notifications.

Examples:

- Successful login
- Invalid password
- Account creation
- Reservation confirmation
- Reservation cancellation

- Favorite updates
- Concierge inquiry submission

This allows the application to communicate state changes without page reloads.

---

37. UI / UX DESIGN

37.1 Visual Theme

The application follows a luxury fine-dining visual language.

Major characteristics include:

- Dark background
- Gold/amber highlights
- Glassmorphism
- Soft shadows
- Large restaurant imagery
- Rounded cards
- Elegant typography
- Subtle animations

---

38. COLOR SYSTEM

The interface primarily uses:

- Near-black backgrounds
- Dark charcoal surfaces
- Amber/gold primary accent
- White primary text
- Muted gray secondary text
- Emerald status colors
- Rose/red cancellation states

This creates a premium restaurant reservation appearance.

---

39. RESPONSIVE DESIGN

The platform is designed for:

- Desktop
- Laptop

- Tablet
- Mobile

Tailwind responsive utility classes are used throughout the HTML.

Typical responsive breakpoints include:

```
sm
md
lg
xl
```

The interface adapts:

- Navigation
- Restaurant cards
- Gallery layouts
- Booking forms
- Dashboard layouts
- Tables
- Modals

---

40. IMAGE ASSET ARCHITECTURE

The project contains 53 WebP image assets.

Assets are stored in:

```
assets/
```

They are used for:

- Restaurant hero images
- Restaurant galleries
- Dining experiences
- User avatars
- Editorial content
- Restaurant interiors
- Food imagery

WebP provides efficient image compression for web delivery.

---

41. ACCESSIBILITY

The project includes several accessibility-oriented practices:

- Semantic HTML
- Image `alt` attributes
- Button labels
- Form labels
- Responsive design
- Icon-based visual cues
- Keyboard-accessible standard HTML controls

Further accessibility improvements are recommended for production.

---

42. ERROR HANDLING

The project contains a dedicated:

```
404.html
```

page for invalid routes or unavailable content.

JavaScript also performs defensive checks before manipulating DOM elements.

---

43. DEPLOYMENT CONFIGURATION

The project includes:

```
.github/workflows/static.yml
```

This indicates GitHub Actions-based static deployment configuration.

The project is therefore suitable for static hosting environments such as:

- GitHub Pages
- Netlify
- Vercel static deployment
- Other static web hosting services

---

44. LOCAL DEVELOPMENT

The README specifies the following local development command:

```
npx serve -p 3000 .
```

The application can then be accessed through:

```
http://localhost:3000
```

---

45. VERSION CONTROL

The project includes a Git repository.

Git is used for:

- Source code management
- Version history
- Branch management
- Remote repository synchronization
- Deployment workflow

Typical commands:

```
git add .

git commit -m "Project Update"

git push origin main
```

---

46. TESTING

46.1 Functional Testing

The following functionality should be tested:

- Home page navigation
- Restaurant discovery
- Search
- City filtering
- Cuisine filtering
- Michelin filtering
- Price filtering
- Sorting
- Restaurant detail page

- Gallery
- Seating selection
- Table availability
- Reservation creation
- Reservation cancellation
- Favorite/hotlist
- Login
- Signup
- Client profile
- Concierge form
- Concierge chat
- Digital VIP pass
- Admin dashboard
- Guest dossier
- Table status
- Restaurant selection

---

47. AUTHENTICATION TESTING

Client

Verify:

- Valid login
- Invalid login
- Registration
- Persistent login
- Logout
- Profile access

Admin

Verify:

- Admin authentication
- Role validation
- Dashboard access
- Reservation management
- Guest dossier

- Operational controls

---

48. RESPONSIVE TESTING

The website should be tested at:

- Desktop resolution
- Laptop resolution
- Tablet resolution
- Mobile resolution

Important areas to verify:

- Navigation
- Restaurant catalog
- Restaurant detail
- Booking modal
- Floor plan
- Reservations
- Admin dashboard
- Tables
- Forms

---

49. BROWSER COMPATIBILITY

Browser	Expected Support
Google Chrome	Yes
Microsoft Edge	Yes
Mozilla Firefox	Yes
Safari	Yes
Mobile Chrome	Yes
Mobile Safari	Yes

The platform relies on modern browser features including:

- LocalStorage
- ES6 JavaScript
- CSS Grid
- Flexbox
- Web APIs

---

50. SECURITY ANALYSIS

The current application is a frontend prototype and should not be considered production-secure.

Current Limitations

- Passwords are stored in LocalStorage.
- Authentication is performed client-side.
- User roles are client-controlled.
- No backend authorization exists.
- No database is connected.
- Reservation data can be modified through browser storage.
- Concierge requests are stored locally.
- No server-side input validation exists.

---

51. PRODUCTION SECURITY REQUIREMENTS

For production deployment, the following should be implemented:

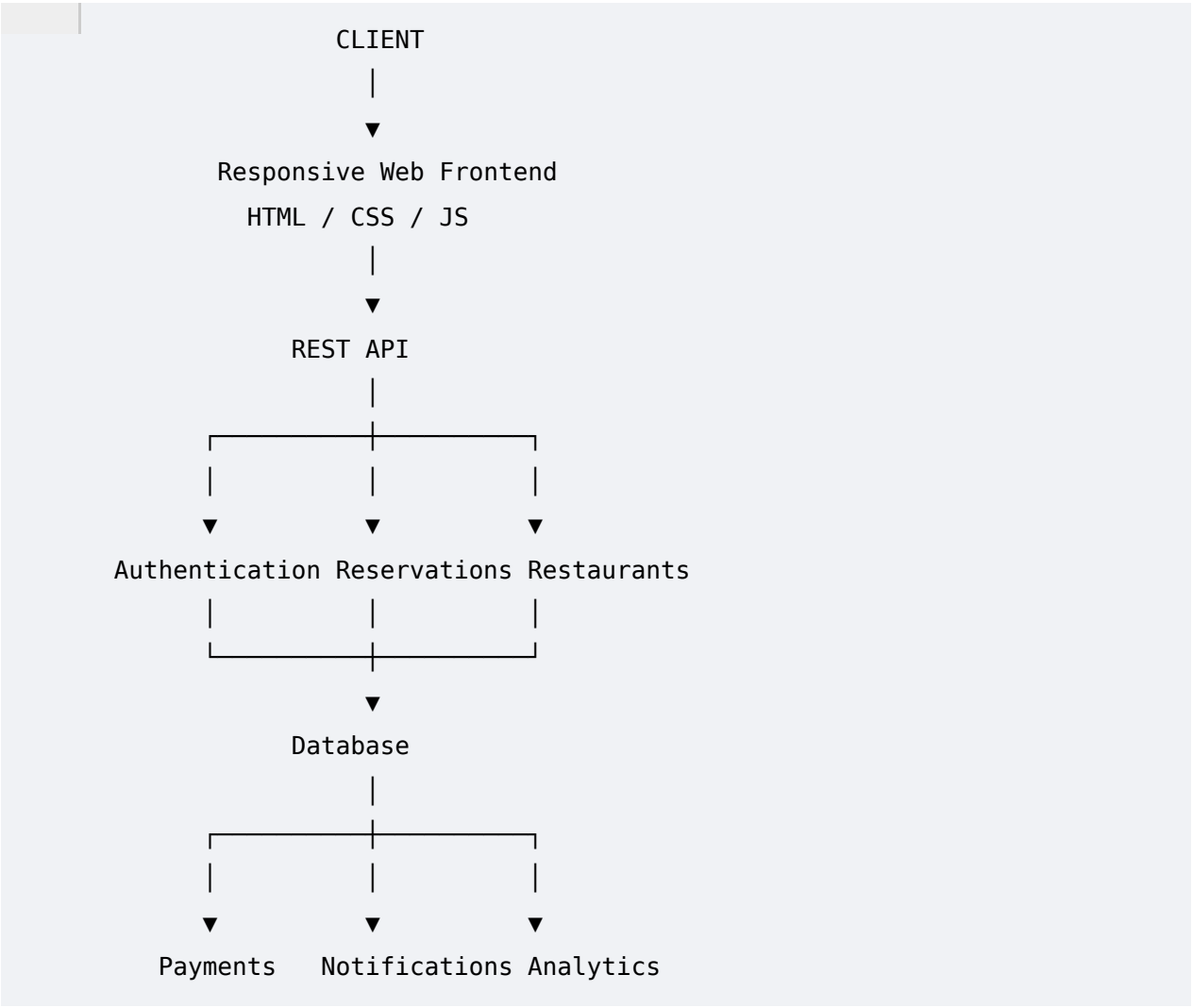
- Backend authentication
- Secure password hashing
- HTTPS
- Secure cookies
- JWT or secure session management
- Server-side authorization
- Role-based access control
- Database validation
- API authentication
- Input sanitization

- Rate limiting
- CSRF protection
- Audit logging
- Secure payment processing

Passwords should never be stored in plain text or LocalStorage in a production environment.

---

52. RECOMMENDED PRODUCTION ARCHITECTURE



---

53. RECOMMENDED DATABASE ENTITIES

A production version could use the following database tables/collections:

```
Users
Restaurants
Chefs
Menus
```

```
    MenuItems
    SeatingAreas
    Tables
    Reservations
    ReservationGuests
    Favorites
    Reviews
    DiningExperiences
    ConciergeInquiries
    Notifications
    Payments
    RestaurantStaff
    RestaurantOperatingHours
```

---

54. FUTURE ENHANCEMENTS

Reservation

- Real-time availability
- Multi-restaurant booking
- Waitlist
- Cancellation policies
- Automated reminders
- Calendar integration

Restaurant

- Real-time menus
- Restaurant owner portal
- Operating hours
- Menu management
- Table inventory

Customer

- Loyalty program
- Dining history
- Personalized recommendations
- Dietary profile
- Saved preferences

Concierge

- AI concierge
- Real human concierge integration
- WhatsApp/SMS integration
- Automated reservation assistance

Payments

- Online deposits
- Credit/debit card processing
- Refund handling
- Cancellation fees
- Digital receipts

---

55. PERFORMANCE RECOMMENDATIONS

For production deployment:

- Optimize WebP images.
- Add lazy loading.
- Minify CSS.
- Minify JavaScript.
- Build Tailwind CSS instead of relying on CDN in production.
- Implement browser caching.
- Use a CDN.
- Reduce unused CSS.
- Reduce unnecessary DOM re-rendering.
- Preload critical assets.
- Optimize large hero images.

---

56. PROJECT STRENGTHS

The project has several strong technical characteristics:

1. Modular JavaScript

Page functionality is separated into individual controllers.

2. Central State Management

`StacklyStore` provides a single state-management layer.

3. Reusable Components

Common navigation, footer, modals, and UI components are centralized.

4. Rich Restaurant Data

Restaurant objects include detailed information covering cuisine, awards, tables, seating, ratings, locations, and availability.

5. Interactive Floor Plan

The restaurant detail page provides a table-selection experience rather than a basic reservation form.

6. Role-Based Operations

The platform differentiates between client and administrative workflows.

7. Persistent Prototype State

LocalStorage enables reservations, accounts, favorites, and inquiries to persist across browser sessions.

8. Premium UX

The design system consistently communicates a luxury fine-dining brand.

---

57. TECHNICAL LIMITATIONS

The current implementation has the following limitations:

- No backend API.
- No real database.
- No real-time restaurant availability.
- No production authentication.
- No production payment gateway.
- LocalStorage-based data persistence.
- Client-side role authorization.
- Static restaurant information.
- Simulated concierge responses.
- Static table availability.
- Static reservation data.
- No actual restaurant partner integration.

---

58. CONCLUSION

Stackly Reserve is a comprehensive frontend restaurant reservation platform designed around luxury dining discovery and reservation management.

The project combines:

- Restaurant discovery
- Advanced filtering
- Michelin dining
- Restaurant profiles
- Interactive floor plans
- Table selection
- Reservation management
- Digital VIP passes
- Favorites/hotlist
- Concierge services
- Client profiles
- Admin operations
- Guest dossiers
- Table status management
- Editorial content
- Responsive design

The application's architecture is well suited for a high-fidelity frontend prototype. Its modular JavaScript controllers, centralized data model, reactive state store, reusable components, responsive Tailwind interface, and LocalStorage persistence provide a strong foundation for future development.

For production deployment, the platform should be extended with a secure backend API, database, authentication service, real-time table availability, payment processing, notification services, and server-side role-based authorization.

---

59. DEVELOPER INFORMATION

Project Name: Stackly Reserve

Application: Luxury Restaurant Reservations & Haute Gastronomy Platform

Developer: Dhamodharan P

Role: Frontend Developer

Frontend: HTML5, CSS3, JavaScript ES6

CSS: Tailwind CSS + Custom CSS

Icons: Lucide Icons

Fonts: Cinzel, Cormorant Garamond, Plus Jakarta Sans

Data Layer: JavaScript Data Objects

State Management: Custom StacklyStore

Persistence: Browser LocalStorage

Assets: WebP

Deployment: GitHub Pages / Static Hosting

Version: 1.0

Document Version: 1.0